UniBot — Project Defense & Viva Master Guide

AI-Powered University Student Support System | Architecture, RAG Pipeline & Cross-Questioning Defense

1. Executive Overview (How to Explain Your Project in 60 Seconds)

In Simple Words: UniBot is an intelligent, 24/7 web-based assistant built for university campuses. Every semester, students struggle with repeated questions about admissions, examination grading, tuition fees, scholarship eligibility, and campus rules. Administrative offices are flooded with inquiries, leading to long queues and delayed responses.

The Solution: UniBot solves this by giving students instant, highly accurate answers grounded in verified university documents. Unlike generic ChatGPT, UniBot does not hallucinate fake fees or rules because it uses a **Retrieval-Augmented Generation (RAG)** pipeline. Additionally, it features a **Dual-Engine Architecture**: it connects to Google Gemini AI for rich conversational responses, but seamlessly falls back to a built-in semantic knowledge synthesizer if the internet drops or API quota runs out.

Component	Technology Used	Key Responsibility
Frontend UI	React 19 + Vite + Vanilla CSS	Responsive student chat, FAQ accordion, notices, admin portal
Backend API	FastAPI (Python 3.13) + Uvicorn	Asynchronous REST endpoints, JWT auth, RAG orchestration
Database	SQLite + SQLAlchemy ORM	Stores users, conversations, messages, FAQs, notices, audit logs
AI / LLM	Google GenAI (Gemini 2.5 Flash)	Context-grounded natural language synthesis with citation linking
RAG Engine	Scikit-Learn (TF-IDF + Cosine)	Sublinear vector indexing, keyword matching & similarity scoring

2. How the RAG Pipeline Works (Step-by-Step)

When an evaluator asks: *'What happens between the student pressing Send and receiving an answer?'*, give this 5-step explanation:

- **Step 1 — Query Ingestion:** The student types a question in React. An authenticated or anonymous request reaches the FastAPI /api/chat/message endpoint.
- **Step 2 — Semantic Retrieval:** The KnowledgeRetriever scans all verified university FAQs and uploaded document text chunks using TF-IDF vectorization and cosine similarity to extract the top 4 most relevant excerpts above a similarity threshold (0.25).
- **Step 3 — Context Prompt Assembly:** The retrieved excerpts, the last 4 messages of conversation history, and strict grounding instructions are assembled into an augmented prompt.

- **Step 4 — Grounded Synthesis:** The prompt is sent to Gemini 2.5 Flash. If Gemini is unavailable, the local fallback_synthesizer uses semantic rule extraction to construct a factual response directly from the retrieved knowledge.
- **Step 5 — Confidence & Audit:** The response is saved to the SQLite database with source citations, similarity score, and latency metrics. The student can submit thumbs up/down feedback.

3. Master Defense: Handling All Cross-Questions

Study these questions carefully. Each includes a simple conversational answer and a technical justification.

Category A: Architecture & Engineering Choices

Q1: Why did you choose FastAPI over Flask or Django?

Simple Answer: FastAPI is modern, extremely fast, and built specifically for asynchronous APIs. Django is too heavy and monolithic for a dedicated chatbot service, while Flask lacks native asynchronous handling and automatic schema validation.

Technical Defense: *FastAPI uses Starlette for high async concurrency (ASGI) and Pydantic v2 for data validation. It generates interactive OpenAPI docs automatically at /docs and delivers throughput comparable to NodeJS and Go.*

■ **Presentation Tip: Highlight that universities experience high traffic spikes during exam results or admissions, where async I/O is critical.**

Q2: Why use RAG instead of fine-tuning an LLM?

Simple Answer: Fine-tuning a model is expensive, time-consuming, and outdated the moment university fees or deadlines change next semester. RAG allows administrators to upload a new PDF or edit an FAQ, and the bot knows the new rule in 1 second without retraining.

Technical Defense: *Fine-tuning alters internal model weights without guaranteeing factual recall. RAG decouples knowledge storage from language generation, eliminating retraining costs and enabling real-time knowledge base updates.*

■ **Presentation Tip: Use the phrase: 'RAG gives us a living knowledge base, not a frozen snapshot.'**

Q3: Why did you build a Dual-Engine architecture (Gemini + Local Synthesizer)?

Simple Answer: If the internet connection fails or the university runs out of API budget, most AI chatbots completely break. Our chatbot automatically detects if the external API fails and switches to an internal semantic synthesizer so students always get their answers.

Technical Defense: *The system wraps external model invocations with a try-catch block and checks gemini_client.is_available(). If generation fails or returns HTTP 429/500, execution falls back seamlessly to the local fallback synthesizer with zero downtime.*

■ **Presentation Tip: Emphasize reliability and fault tolerance — evaluators love systems with built-in disaster recovery.**

Category B: AI, Retrieval & Hallucination Prevention

Q4: How do you prevent hallucinations (bot making up fake numbers or rules)?

Simple Answer: We constrain the model with strict system instructions: it is strictly forbidden from guessing. It is instructed to answer ONLY from the verified university context. If the context does not contain the answer, it gives a polite referral to the Registrar's Office.

Technical Defense: *Grounding is enforced via two layers: prompt boundary injection (STRICT GROUNDING GUIDELINES) and retrieval gating (similarity threshold of 0.25). Furthermore, if similarity is low, fallback_triggered is flagged in the query log for admin review.*

■ **Presentation Tip: Mention that accuracy is far more important than creativity in university policies.**

Q5: Why did you use TF-IDF and Cosine Similarity for the retriever instead of a vector database like Pinecone or Chroma?

Simple Answer: TF-IDF is lightweight, instantaneous to compute, runs completely offline, and requires zero external cloud hosting fees or heavy dependencies. For a university knowledge base with hundreds of policy documents, TF-IDF + Cosine Similarity provides sub-millisecond lookups with exact keyword preservation.

Technical Defense: *We configured TfidfVectorizer with sublinear_tf=True and ngram_range=(1,2) to capture domain bigrams (e.g., 'semester fee', 'passing grade'). We also added an exact keyword boosting heuristic that prioritizes title and category matches.*

■ **Presentation Tip: Add: 'Our retriever architecture is modular — we can swap TF-IDF for ChromaDB or pgvector by simply replacing the retriever class.'**

Q6: How does the bot maintain conversation history?

Simple Answer: When a student asks follow-up questions like 'What about for SC/ST category?', the bot remembers the prior context because we store conversations in the database and feed the last 4 turns into the LLM prompt.

Technical Defense: *The Message table records sender and content linked to a conversation_id. In chat.py, past messages are ordered by timestamp and passed as structured history to Gemini or the local synthesizer.*

Category C: Document Processing & Administration

Q7: How do you process uploaded PDF documents in the admin dashboard?

Simple Answer: When an admin uploads a circular or policy handbook PDF, our system extracts the text page-by-page using PyPDF and splits large text into overlapping chunks so the retriever can pinpoint the exact paragraph.

Technical Defense: *The text extractor uses sliding window chunking (approx 600-800 characters with 100 character overlap). Each DocumentChunk is saved in the database with its document_id and page_number, enabling precise citations.*

■ **Presentation Tip: Mention that students see citations like 'Academic Handbook (Page 4)' in their chat answers.**

Q8: What happens when a student asks something the bot does not know?

Simple Answer: The system detects low similarity or fallback activation, answers politely with contact details, and records the question into the Unanswered Queries table. The administrator can view this list on their dashboard and add the missing FAQ with one click.

Technical Defense: *This forms a continuous feedback loop: student queries highlight knowledge gaps, admins populate new answers, and the system continuously grows smarter without code changes.*

■ **Presentation Tip: Evaluators love this — call it an 'Active Learning / Feedback Loop'.**

Category D: Security, Database & Scalability

Q9: How is security and authentication handled?

Simple Answer: We use industry-standard JSON Web Tokens (JWT). Passwords are never stored as plain text — they are salted and hashed using bcrypt. Sensitive admin endpoints are protected by role-based guards.

Technical Defense: *FastAPI dependencies (get_current_user and get_current_active_admin) extract and verify the bearer token signature on protected routes. Unauthorized users receive HTTP 401/403 errors.*

■ **Presentation Tip: Confirm that guest students can still chat without logging in, reducing friction for quick inquiries.**

Q10: Why SQLite? Can this scale to thousands of students?

Simple Answer: SQLite was chosen for local development and self-contained zero-configuration portability. However, because we use SQLAlchemy ORM, switching to PostgreSQL or MySQL in production requires changing just ONE line in .env (DATABASE_URL) without rewriting any queries.

Technical Defense: *SQLAlchemy abstracts all DDL and queries. In a production deployment with Docker and Kubernetes, the backend connects to an enterprise PostgreSQL cluster with connection pooling.*

■ **Presentation Tip: State clearly: 'SQLite is our development driver; the code is 100% database agnostic.'**

Q11: How did you prevent API keys from leaking to GitHub?

Simple Answer: All secrets, tokens, and database files are isolated in .env and .gitignore. When we initialized and pushed the repository, git was verified to stage only clean source code. The repository contains a sanitized .env.example template for new developers.

Technical Defense: *We implemented comprehensive .gitignore rules covering **/.env, *.db, .venv/, and node_modules/.*

4. Quick Presentation Checklist & Demo Flow

1. **Introduce the Problem:** State that university information is fragmented across circulars, websites, and notice boards, wasting hours for both students and staff.
2. **Live Demo (Student Chat):** Ask a natural question: *'What is the minimum attendance requirement to sit in semester exams?'* Show the instant answer, confidence score, and source citation.
3. **Demonstrate Multi-turn Memory:** Ask a follow-up: *'What happens if I have medical leave?'* Show how it understands context.
4. **Show the Fallback / Resiliency:** Explain how the dual engine protects against internet or quota drops.
5. **Show the Admin Dashboard:** Log in as admin, show real-time query analytics, upload a notice, and show the unanswered query resolver.
6. **Conclude with Future Scope:** Mention voice input (speech-to-text), multi-language regional support (Hindi/Tamil/Spanish), and WhatsApp/Telegram bot integrations.